

Parte 6 - ASP.NET Core e Web API

Architettura di ASP.NET Core

ASP.NET Core è la naturale evoluzione per l'era dei microservizi: open-source, multiplatforma (cross-platform) e modulare. Non carica librerie superflue, ma solo i pacchetti NuGet necessari, risultando molto leggero.

La sua architettura è a strati:

- **Host:** avvia il web server integrato (Kestrel), gestisce le richieste HTTP e avvia la Dependency Injection (DI) e il Logging. Crea le tre classi fondamentali per ogni interazione: `HttpRequest`, `HttpResponse` e `HttpContext`.
- **Middleware:** una "pipeline" di componenti che elabora la richiesta in sequenza.
- **Routing:** mappa l'URL della richiesta verso l'endpoint che dovrà gestirla.
- **Endpoint:** il codice che esegue il lavoro reale (es. Web API, MVC, Razor Pages).

Ambiente di Sviluppo e Progetto

Lo sviluppo moderno non richiede IDE pesanti, ma si basa sul **.NET SDK** (compilatore e runtime) e sulla sua **CLI** (Command Line Interface).

- **Comandi chiave (dotnet):** `new` (crea un progetto da un template, es. `webapi`), `build` (compila il codice in assembly `.dll`), `run` (compila ed esegue l'app).
- **Compilazione:** `debug` (crea i file `.pdb` per collegare l'eseguibile al codice sorgente) e `release` (ottimizza il codice e rimuove i simboli di debug per la produzione).
- **Il file .csproj:** è un file XML che governa il progetto, indicando la versione del framework e i riferimenti ai pacchetti NuGet e ad altre librerie interne.

Configurazione Base: Program.cs e Middleware

Nelle versioni recenti (da .NET 6), il `Program.cs` usa i *Top Level Statements* (senza una classe `Main` esplicita). Il bootstrap dell'app avviene in due step:

1. **Configurazione dei Servizi (`WebApplication.CreateBuilder`):** registra le dipendenze nel contenitore DI (tramite `builder.Services`).
2. **Configurazione della Pipeline (`builder.Build()`):** definisce in quale ordine i **Middleware** intercettano la richiesta HTTP usando `app.Use...`.
 - *Nota sui Middleware:* possono elaborare la richiesta e chiamare il componente successivo (`next`), oppure fermare la catena restituendo subito una risposta. L'ordine è rigido (es. `ExceptionHandler` va all'inizio per catturare le eccezioni globali).

Dependency Injection (DI)

Il pattern DI serve a disaccoppiare il codice (Inversion of Control). Le classi non istanziano le proprie dipendenze, ma le richiedono nel costruttore tramite interfacce; il framework inietta l'implementazione concreta.

I servizi si registrano con tre diversi **Service Lifetime**:

- **AddSingleton:** Un'unica istanza creata al primo utilizzo e condivisa per tutta la vita dell'app. (Attenzione alla concorrenza/multithreading).
- **AddScoped:** Una nuova istanza creata *per ogni singola richiesta HTTP*. Ideale per mantenere lo stato di una specifica transazione web.
- **AddTransient:** Una nuova istanza generata *ogni volta* che il servizio viene richiesto. Ideale per servizi leggeri o in scenari multithreading.

Progettazione Web API e HTTP

Le Web API espongono servizi sul web. Si dividono principalmente in:

- **SOAP:** Protocollo formale e rigido basato su XML e contratti espliciti (WSDL/XSD).
- **REST:** Pattern architetturale che sfrutta la natura del protocollo HTTP. È leggero, solitamente usa JSON come formato dati, ed è lo standard di fatto per i microservizi.

Comunicazione RESTful:

- **Verbi HTTP (Azioni):** `GET` (lettura), `POST` (creazione), `PUT` (aggiornamento totale), `PATCH` (aggiornamento parziale), `DELETE` (eliminazione).
- **Status Code (Esito):**
 - *2xx (Successo):* 200 OK, 201 Created.
 - *4xx (Errore Client):* 400 Bad Request (dati non validi), 401 Unauthorized (credenziali mancanti), 403 Forbidden (permessi mancanti), 404 Not Found.
 - *5xx (Errore Server):* 500 Internal Server Error, 503 Service Unavailable.

Controller e Dati

Nelle API di ASP.NET Core, la logica risiede nei **Controller** (derivati da `ControllerBase` e marcati con `[ApiController]`). L'attributo garantisce comportamenti automatici (es. la restituzione di un 400 Bad Request se i dati della richiesta sono malformati) e forza l'uso del routing tramite attributi.

- **Routing Dichiarativo:** Si mappa l'URL ai metodi usando attributi come `[Route("[controller]")]` sulla classe e `[HttpGet("{id}")]` sui metodi (action).
- **Model Binding:** Il processo automatico che converte i dati JSON o URL in parametri C#. Usiamo attributi come `[FromBody]` (corpo JSON), `[FromQuery]` (parametri URL) e `[FromRoute]` (segmenti dell'URL) per mapparli.
- **Asincronia:** I metodi dovrebbero sempre usare `async/await` e restituire `Task<IActionResult>`. Questo evita che il server web si blocchi in attesa del database, aumentando drasticamente la scalabilità sotto carico.

Tool e Pratiche Avanzate

- **DTO e AutoMapper:** Non si espongono mai i model di dominio direttamente. Si usano i DTO (Data Transfer Object) e la libreria AutoMapper per convertire automaticamente i Model in DTO (e viceversa) in base a profili configurati.
- **Configurazioni Fortemente Tipizzate:** I parametri (es. stringhe di connessione) stanno nel file `appsettings.json`. Con `builder.Services.Configure<T>`, vengono iniettati nei controller come oggetti di tipo `IOptions<T>`.

- **Logging:** Il framework inietta nativamente un `ILogger` nei servizi per tracciare eventi e difetti (es. `_logger.LogInformation()`), essenziale per il debug di un microservizio distribuito.